

06/07/2026 TP2 : Compteur de temporisation.

L'objectif de cet TP est faire clignoter une LED en utilisant un compteur de temporisation. Il permet de compter le nombre de coup d'horloge nécessaire pour attendre un temps voulu. En connaissant la fréquence de l'horloge il est possible de déterminer combien de périodes d'horloge il faut compter pour attendre un certain nombre de seconde.

1. L'horloge du système est fixée à 100MHz. Combien de période faut-il compter pour attendre 2 secondes ? Combien de bits faut-il au minimum pour représenter cette valeur ?

$$nb_{horloge} = \text{delai} * f_{Horloge} = 2 * 100E6 = 200E6$$

On divise le délai d'attente par la période de l'horloge. Il est nécessaire de compter jusqu'à 200E6 pour attendre 2s. Le nombre de valeur prise par un nombre de bit est donné par la formule suivante :

$$V_{max} = 2^n$$

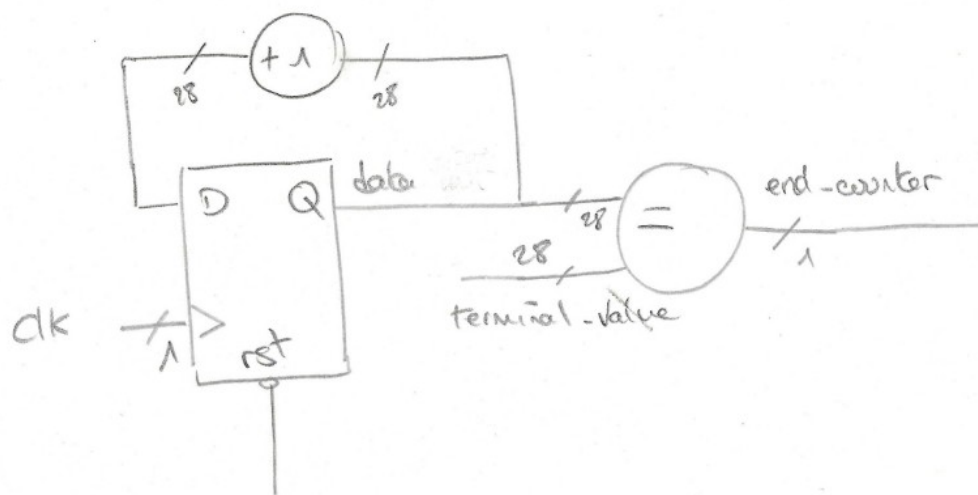
avec n le nombre de bit. On résout l'équation suivante :

$$2^n = 200E6 \Rightarrow n = \frac{\log(200E6)}{\log(2)} = 27,6$$

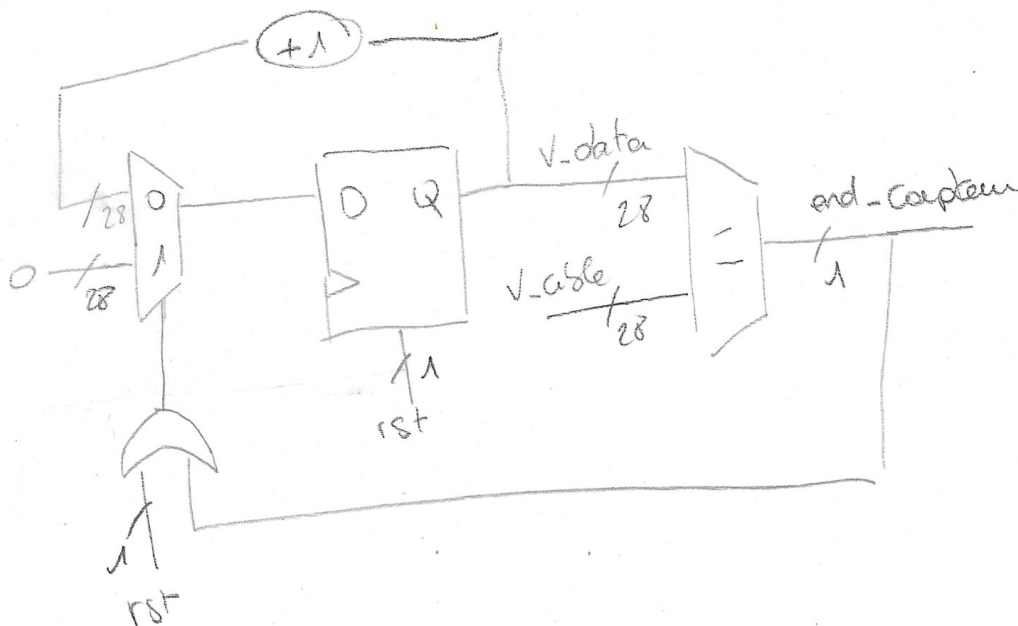
en arrondissant à l'entier supérieur, il faut donc 28 bits pour compter jusqu'à 200E6, 28 bits permettent de compter jusqu'à 268 435 456.

$$2^{28} = 268\,435\,456$$

2. Dessinez le schéma RTL de ce compteur. Si le compteur atteint la valeur calculée précédemment, un signal `end_counter` passe à 1, sinon `end_counter` vaut 0. N'oubliez pas de mettre sur chaque signal son nombre de bits. Commencez par réaliser une boucle d'incrémentation : +1 à chaque coup d'horloge.



3. Ajoutez une condition pour que le compteur soit remis à 0 lorsqu'il a atteint la valeur souhaitée.



On rajoute un multiplexeur permettant de sélectionner l'entrée de la bascule de stockage du vecteur V_data . $end_compteur$ est actif pendant un coup d'horloge ce qui réinitialise le compteur. La porte ou permet d'avoir le même effet lorsque le reset est actif.

4. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture.

Signal d'entrée :

Clk signal d'horloge,

resetn permet de remettre à zéro le compteur et la sortie *end_counter*.

Signal de sortie :

end_counter Signal indiquant que la valeur cible à été atteinte.

signal interne :

v_data Vecteur de 28 bits incrémenté à chaque coup d'horloge,

v_cible Vecteur 28 bits contenant la valeur cible fixant le délai,

S_end_counter Signal indiquant que la valeur cible à été atteinte.

On ne peut lire des sorties ni écrire des entrées, il faut les affecter à des signaux internes.

5. Ecrivez à présent le compteur en VHDL en suivant le schéma RTL, faites attention de bien faire correspondre les noms des signaux de votre code VHDL avec ceux de votre schéma RTL.

6. Ecrivez un fichier de testbench pour tester votre design

Le fichier de *testbench* permettra de vérifier que l'horloge incrémente correctement *v_data*,

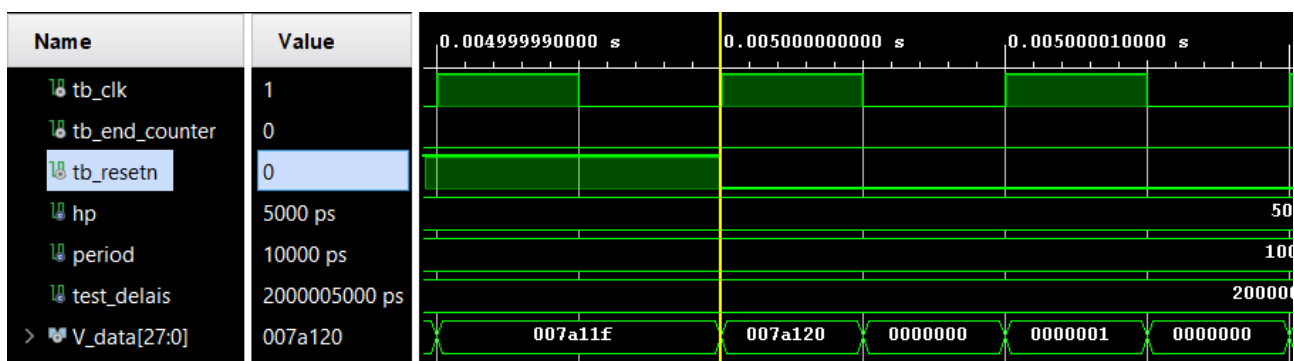
que le signal *end_compteur* passe bien à l'état haut après deux secondes, ainsi que le signal *resetn* remette à 0 *V_data*.

Par convention, on utilise *resetn* pour un reset actif sur état bas *resetn* = reset not.

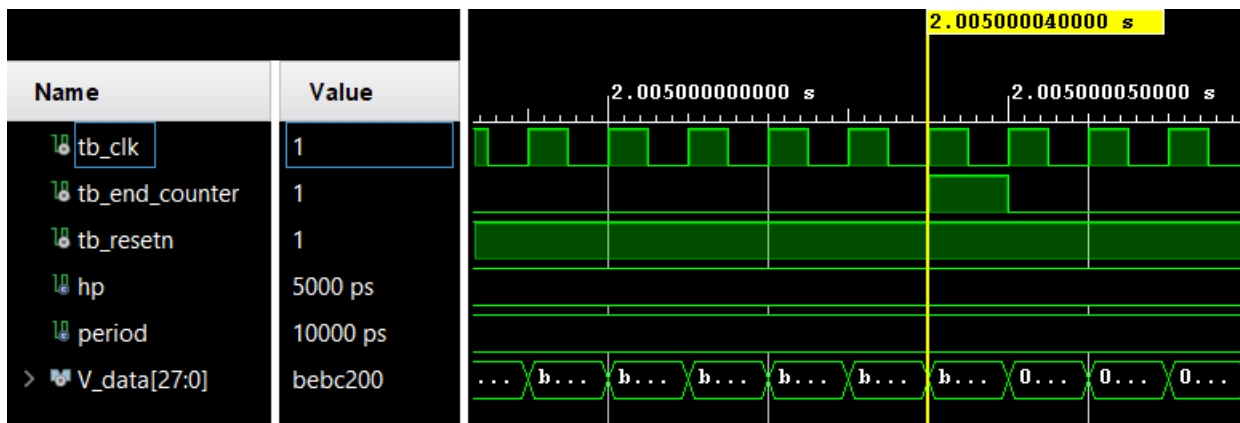
7. Lancez une simulation. Que devez-vous observer sur votre chronogramme pour vérifier que votre design est valide ?

Sur le chronogramme, je dois observer une mise à l'état haut du signal *end_compteur* après 2 secondes de test. On remarque que le vecteur *V_cible* est bien initialisé et que le vecteur *V_data* s'incrémente correctement à chaque coup d'horloge.

L'état haut du signal *end_compteur* doit être levé exactement après avoir atteint les deux secondes, pour cela, il faut que le signal d'horloge commence par un front montant, attention je commence à compter au premier coup d'horloge donc pour compter 200 coup, il faut que j'aille de 0 à 199.



Ici le vecteur *V_data* est bien ramené à zéro lorsque le signal *resetn* est à 0 (reset not actif sur état bas).

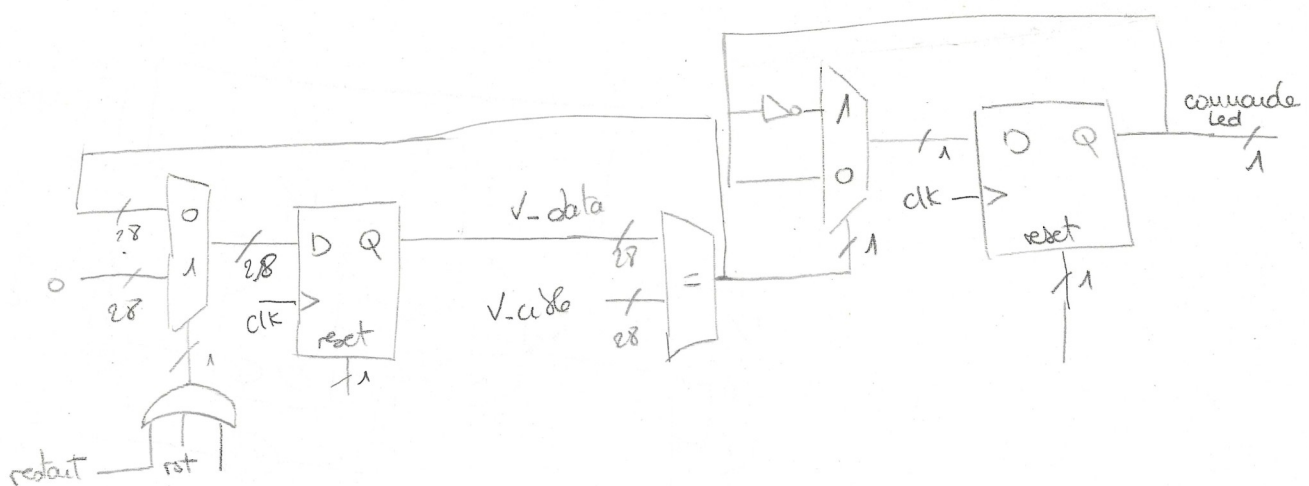


Le signal *end_counter*, passe bien à l'état haut pendant une période d'horloge, après deux secondes, et 50ms et 50ns car le reset est maintenu pendant 50ns après 50 ms de simulation dans le testbench.

8. Associez une LED avec le signal de test d'arrêt du compteur. Pour cela, il faudra ajouter une sortie et la relier à une broche d'une LED dans le fichier de contrainte (.xdc). La LED sera alors allumée pendant seulement un coup d'horloge

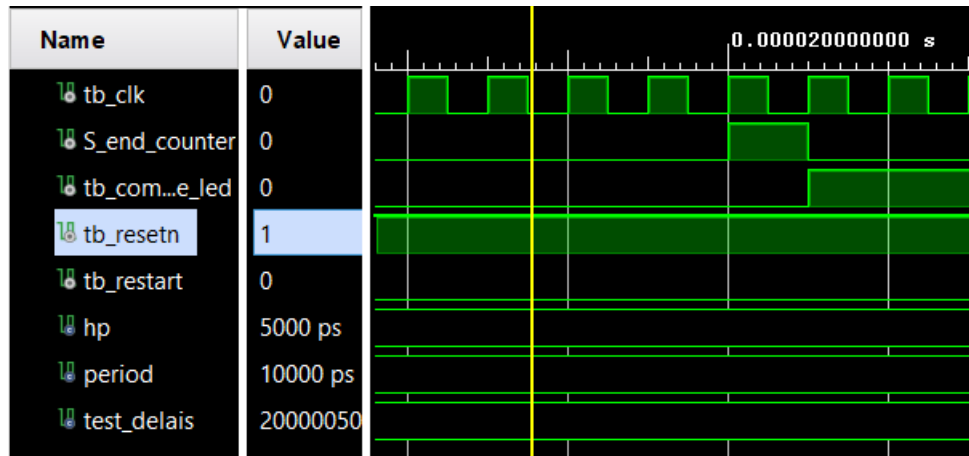
Modification du fichier de contrainte en décommentant une ligne dédiée à une led et utilisation de la variable *end_compteur* pour la piloter.

9. Modifiez le schéma RTL du compteur pour ajouter une remise à 0 lorsqu'un signal restart est à 1. Ajoutez la logique nécessaire pour que la LED clignote telle que : allumée 2s, éteinte 2s.

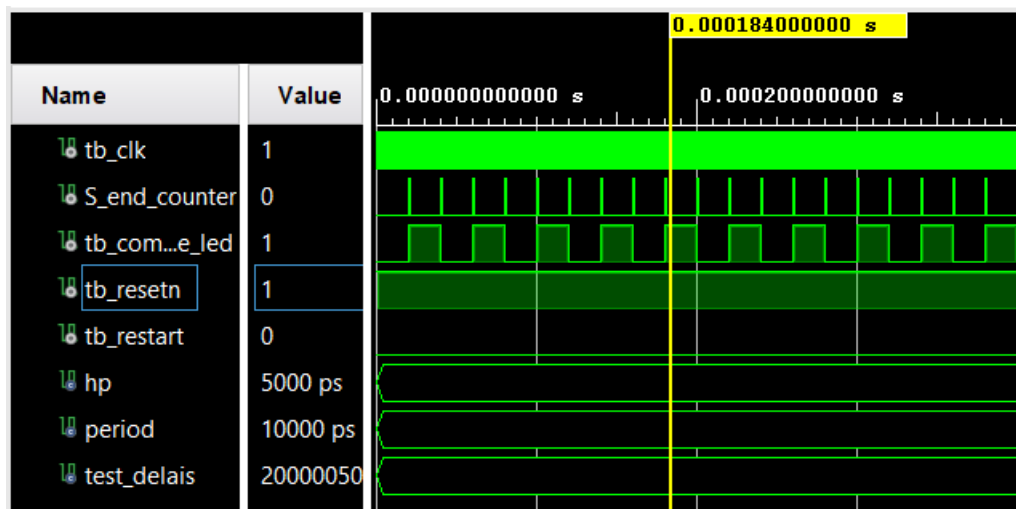


le signal *end_compteur* à la sortie de la comparaison entre *V_data* et *V_cible* pilote désormais un multiplexeur permettant de choisir soit de maintenir l'état de la bascule de sortie soit de l'inverser.

10. Faites les mises à jour nécessaires sur le code VHDL pour correspondre au nouveau schéma. Le signal *restart* sera une entrée du design



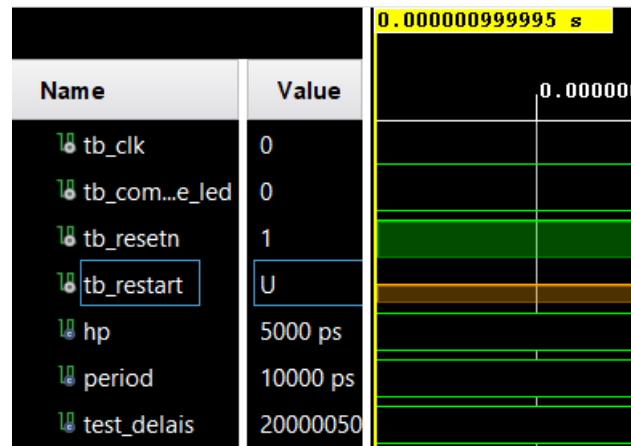
Lorsque *S_end_counter* est passé à l'état haut, l'état de commande led est inversé (*tb_commande_led*).



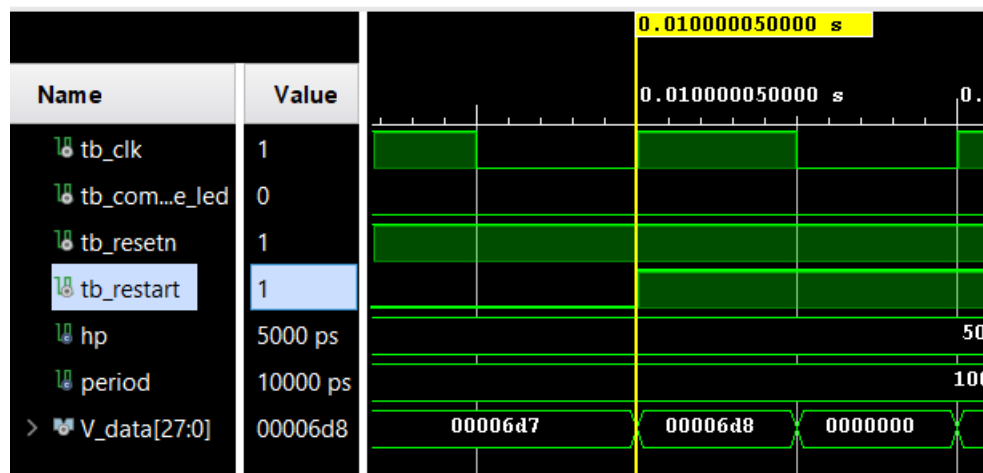
11. Associez la nouvelle entrée restart à un bouton.

12. Mettez à jour votre testbench puis vérifiez votre design avec une simulation. Quels sont les signaux que vous devez observer ?

Les signaux que je dois observer sont l'horloge, le restart, le reset, la sortie *commande_led* ainsi que le signal interne *S_endcompteur*.



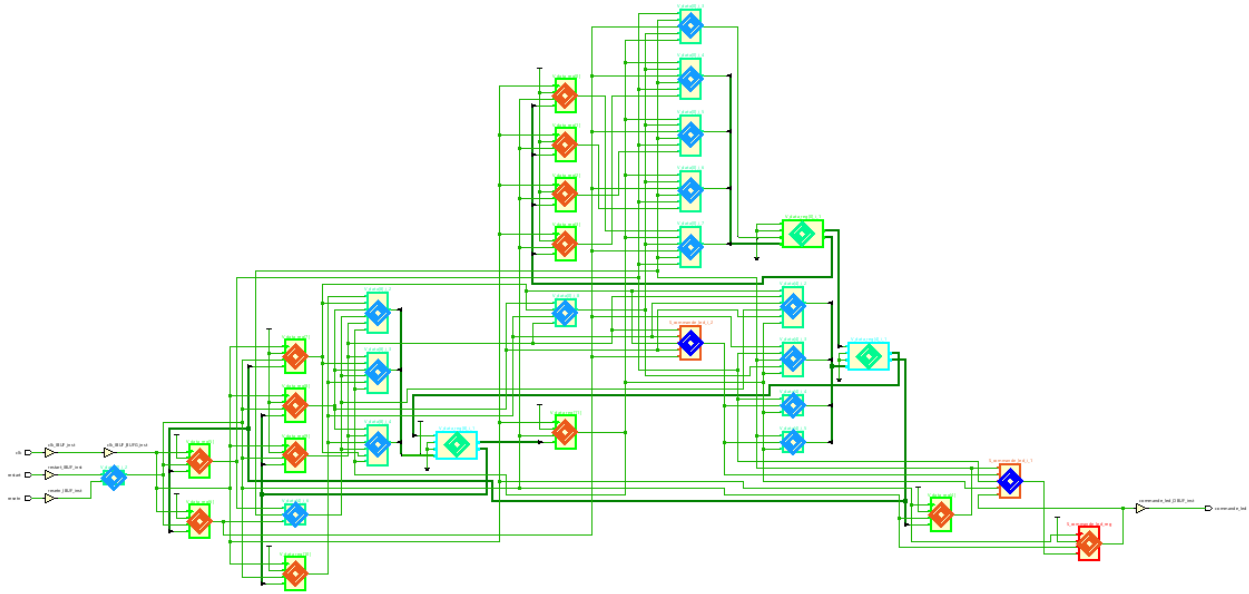
Si l'entrée `tb_restart` n'est pas initialisé à 0, elle se met dans un état indéfini qui empêche la simulation.



`V_data` se réinitialise lorsque `tb_restart` est actif.

13. Exécutez la synthèse puis ouvrez la schématique. Identifiez sur la schématique les différents éléments de votre architecture RTL.

(Schématique généré avec un délais de temporisation de 2000 coup d'horloge. Il y à donc moins d'éléments que dans l'implémentation finale)



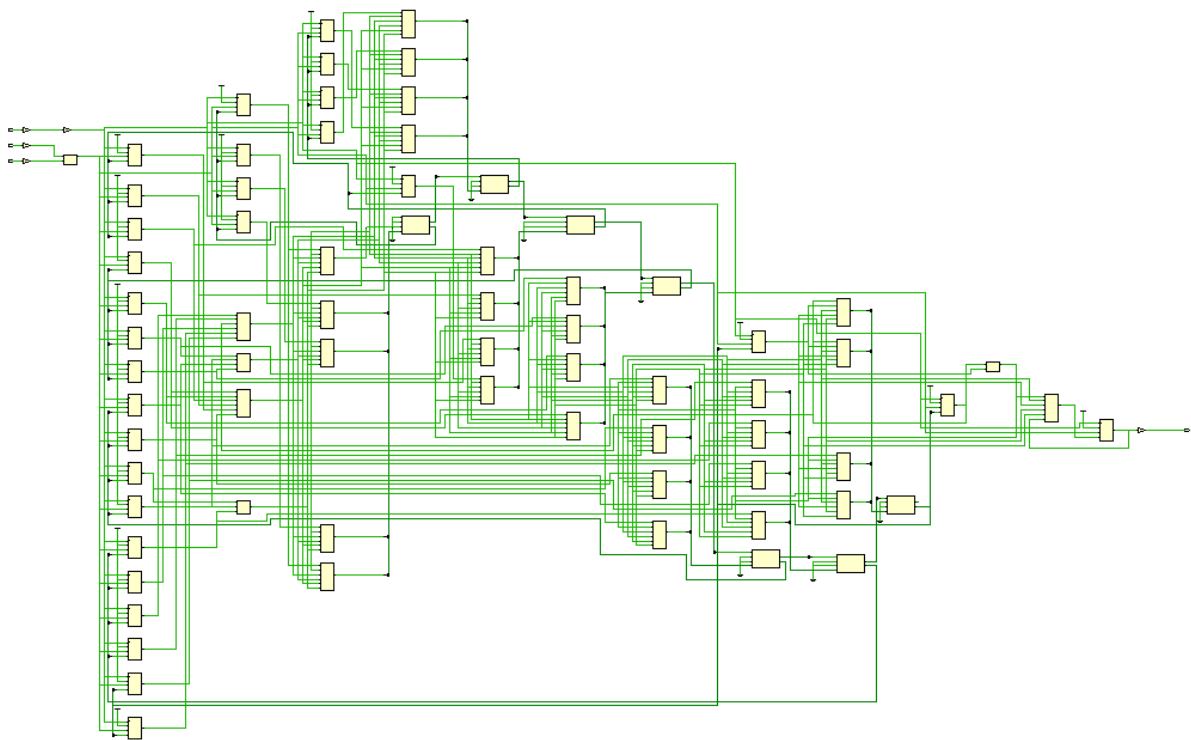
Marque rouge : Registres permettant la mémorisation

Marque verte : Sommateurs

Marque bleue : LUTS.

Les éléments surlignés en vert servent à V_data , ceux en rouge à $S_commande$.

En bas à gauche on observe les trois entrées, *reset*, *restart* et l'horloge, en bas à droite le signal de contrôle de la diode.



Schématique d'un compteur de temporisation 2sec.

14. Ouvrez le rapport de synthèse et relevez les ressources utilisées. Comparez vos résultats avec les résultats attendu selon votre architecture RTL

Report Cell Usage:		
	Cell	Count
1	BUFG	1
2	CARRY4	7
3	LUT2	2
4	LUT3	1
5	LUT4	1
6	LUT6	32
7	FDCE	29
8	IBUF	3
9	OBUF	1

Trois buffers d'input pour trois entrée (Reset, restart et clock), un buffer de sortie (commande_led).
 Il y a 7 sommateurs 4 entrées. On retrouve là les 28 bits nécessaire à compter jusqu'à 200 000 000.
 Il y a 29 bascules de mémorisation.

15. Ouvrez le Set Up Debug. Placez des sondes sur les signaux à observer que vous avez défini à la question 12

16. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

```

Max Delay Paths
-----
Slack (MET) :      26.243ns (required time - arrival time)
  Source:        dbg_hub/inst/BSCANID.u_xsdbm_id/SWITCH_N_EXT_E
                  (rising edge-triggered cell FDRE clocked by
  Destination:   dbg_hub/inst/BSCANID.u_xsdbm_id/SWITCH_N_EXT_E

-----
| Design Timing Summary
| -----
-----

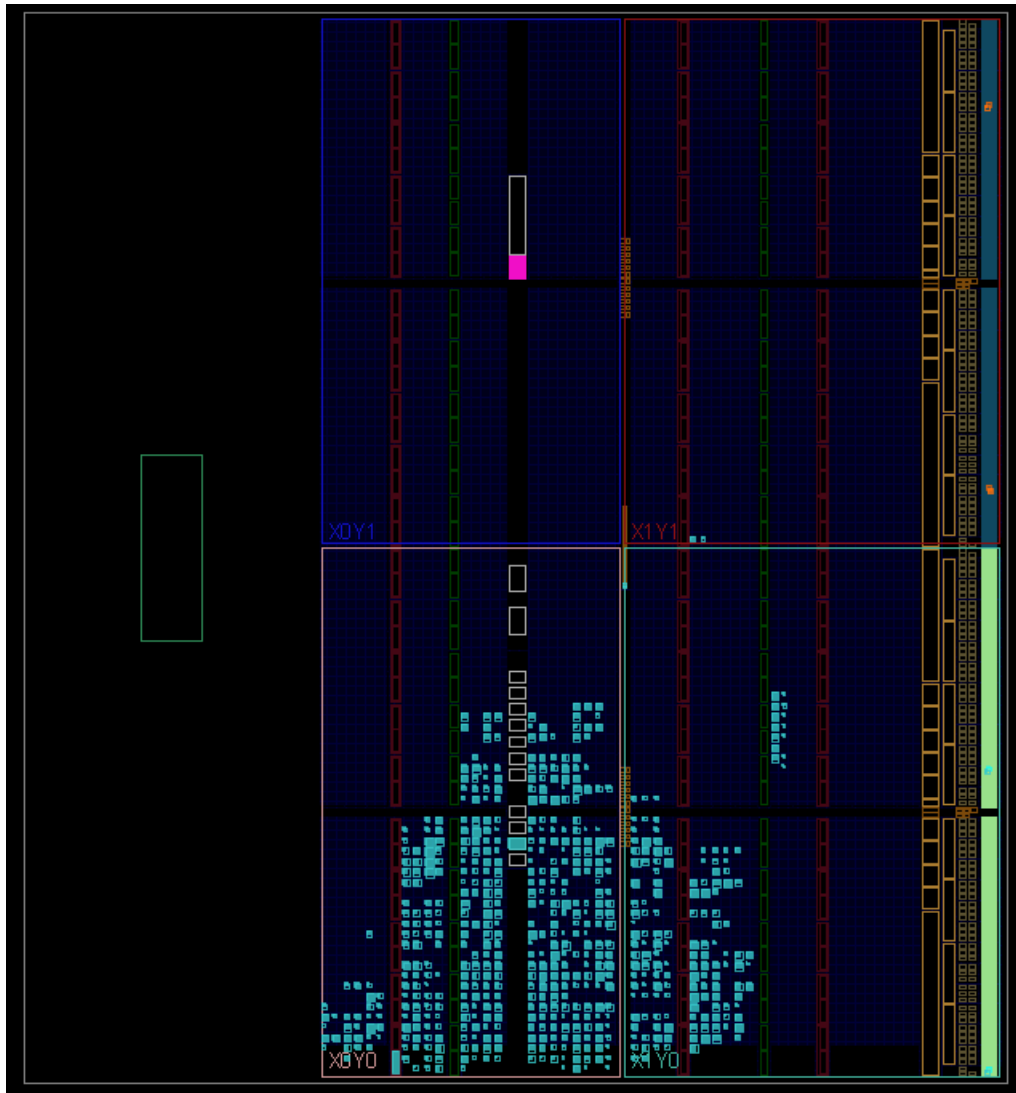
WNS (ns)      TNS (ns)  T
-----      -
26.243        0.000

WHS (ns)      THS (ns)
-----      -
0.049         0.000
    
```

Chemin critique (ns)	26.243
Worst negative slack (ns)	26.243
Total negative slack (ns)	0
Worst hold slack (ns)	0.049
Total hold slack (ns)	0

La total negative slack et le total hold slack sont à 0, cela signifie que le temps de transit au travers des portes logiques est inférieur à la période de l'horloge.

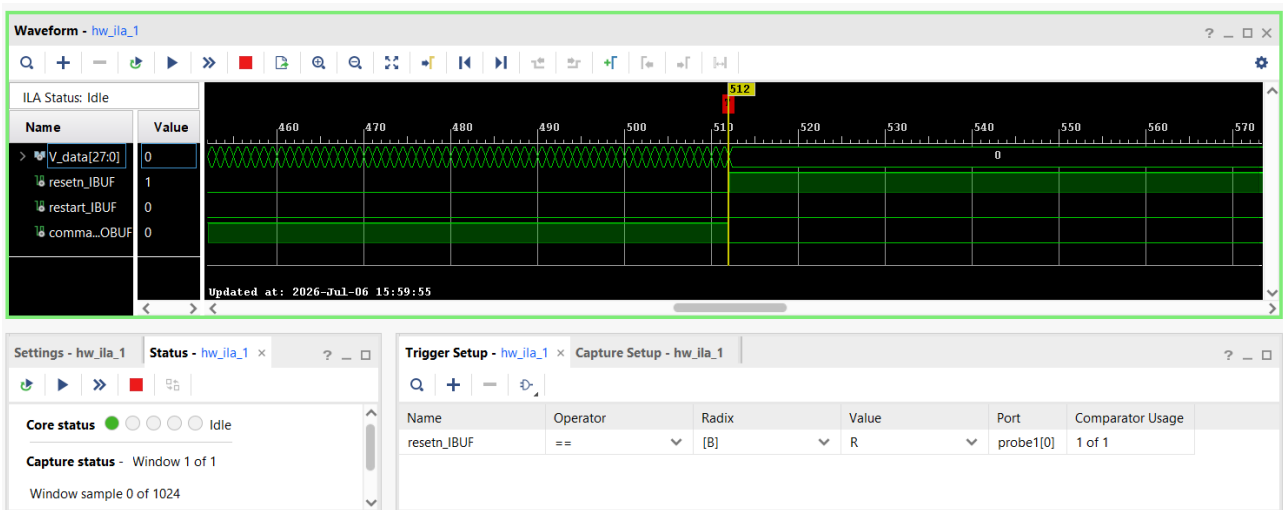
Comment expliquer que le chemin critique à un délai supérieur à la période de l'horloge ? (10ns).
Le chemin critique passe par plusieurs bascules dont l'horloge est une entrée.



Allocation des luts sur le FPGA.

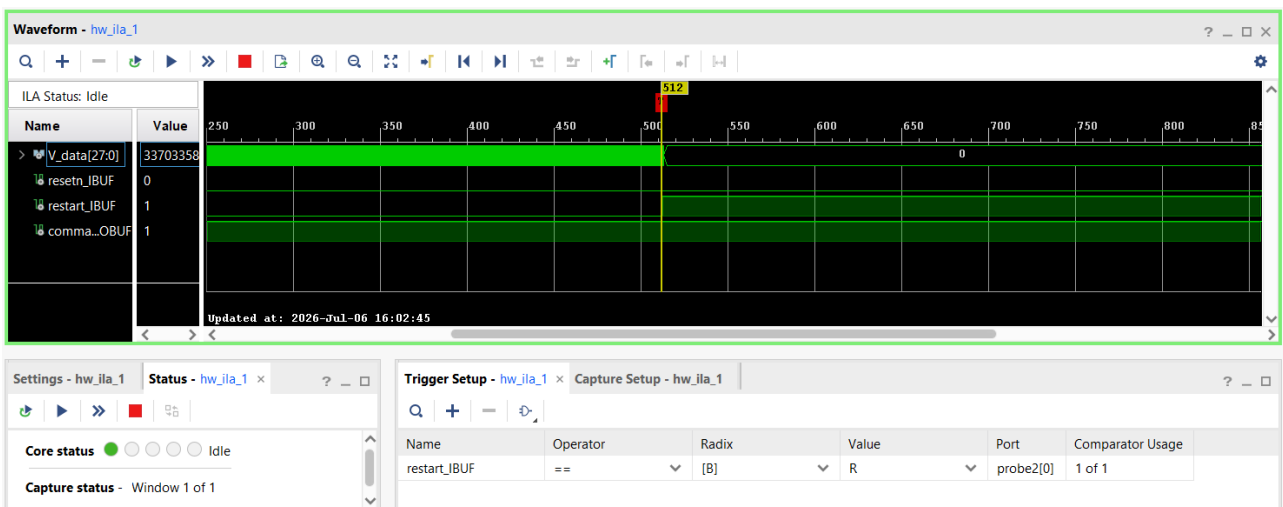
17. Générez le bitstream pour observer le système sur carte et le résultat de la ILA

Test ILA du signal de reset. Trigger actif sur front montant.



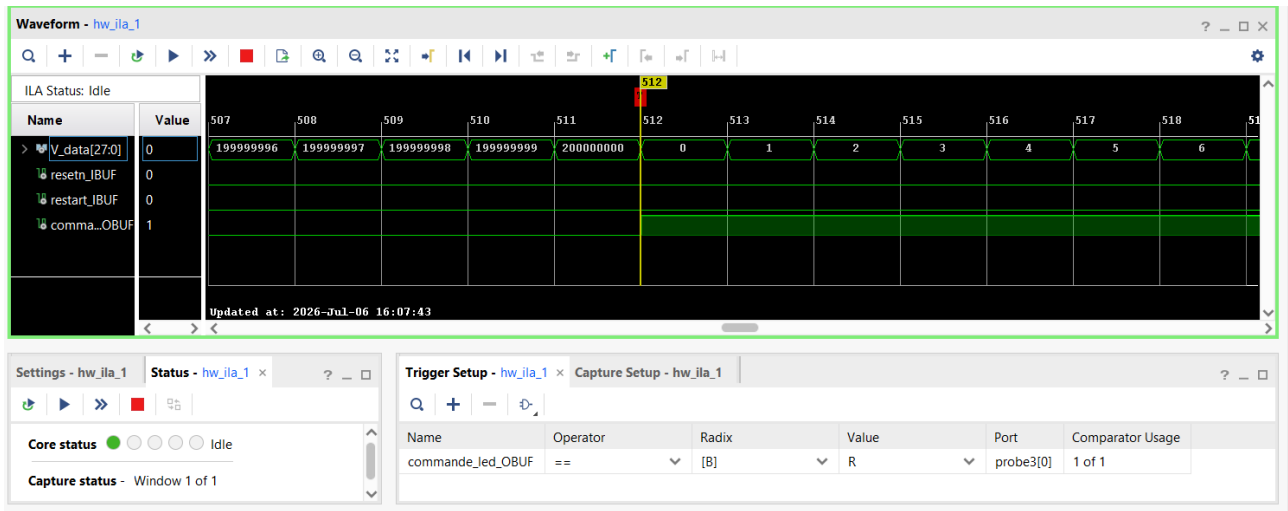
On constate que V_data est bien remis à zéro et que $commande_led$ est mis à l'état bas.

Test ILA du signal de restart. Trigger actif sur front montant.



On constate que V_data est bien remis à zéro mais que l'état de $commande_led$ est maintenu.

Test ILA du signal de commande_led. Trigger actif sur front montant.



Le signal *commande_led* change d'état lorsque le vecteur compteur *V_data* atteint 200 000 000 coup d'horloge, il est ensuite remis à 0.

Sur la carte, on voit clignoter la led 0 en bleue 2s d'intervalle.

Remarque sur des bonnes pratiques :

- 1/ Eviter les instructions compétitives dans la partie séquentielle, On évite de mettre plusieurs Ifs qui viennent concurrencer l'horloge, on évite toutes les indéterminations.
- 2/ l'entrée clock d'une bascule doit être dédiée au signal d'horloge, **pas de logique** sur une horloge.